

Sommario

Capitolo 5 – File I/O	2
System Call e Programmazione di Sistema	2
Gestione dei file.....	2
Chiamata di sistema open.....	2
Chiamata di sistema creat.....	4
Chiamata di sistema close	4
Chiamata di sistema lseek	4
Esecuzioni mostrate:	6
Analisi passo per passo:	6
Chiamata di sistema read	8
Chiamata di sistema write	9
Condivisione di file.....	10
Operazioni atomiche	11
Duplicare i descrittori di file	11
Le system call dup e dup2	12
Gestione degli errori: perror() e errno.....	13
Capitolo 6 – File e Directory.....	18
System call stat, fstat e lstat	18
La struttura stat.....	19
Tipi di file	19
User ID e Group ID.....	21
Set User ID e Set Group ID.....	21
Set user-ID	21
Set group-ID	22
Permessi di accesso ai file	22
Controllo dei permessi da parte del kernel	23
Significato dei permessi in base al tipo di oggetto	23
Sticky bit.....	24
Meccanismi avanzati di identificazione: saved UID e GID.....	24
System call access	24
Proprietà dei nuovi file e directory	25
Permessi e creazione dei file: system call umask.....	26
System call chmod e fchmod	27
System call chown, fchown e lchown	29
Dimensione dei file.....	29
Troncamento: System call truncate() e ftruncate()	30
File System.....	30
Struttura e organizzazione	31
Numero di link sulle directory	31
I-node.....	32
Link hard e simbolici	32
System call link() e unlink()	33
System call symlink() e readlink()	33
System call mkdir() e rmdir()	34
Letture delle directory	34
System call chdir() e funzione getcwd()	35

Capitolo 5 – File I/O

System Call e Programmazione di Sistema

Nel contesto della **programmazione di sistema** UNIX, le **system call** costituiscono l'interfaccia diretta tra i programmi utente e il **kernel**, ovvero il nucleo del sistema operativo. Attraverso esse, i processi possono accedere ai **servizi fondamentali** del sistema, come la **gestione dei file**, la **creazione di processi** e la **comunicazione interprocesso**. A differenza delle semplici **funzioni di libreria**, le system call attivano direttamente una routine interna del kernel, permettendo l'esecuzione di operazioni privilegiate.

L'invocazione di una system call, come **open**, comporta il passaggio di argomenti tramite i **registri della CPU** e l'attivazione di un'istruzione speciale (una **trap**) che trasferisce il controllo al kernel. Tuttavia, non tutte le funzioni utilizzate nello sviluppo software coinvolgono direttamente il sistema operativo: molte fanno parte della **libreria standard del C**, che fornisce sia un'interfaccia semplificata alle system call, sia **funzionalità autonome** eseguite esclusivamente nello **spazio utente**.

Funzioni come **printf** incapsulano system call (nel caso specifico, **write**), mentre altre, come **strcpy** o **atoi**, operano senza accedere al kernel. Altre ancora, come **malloc**, si basano su system call sottostanti, come **sbrk**, per gestire la memoria. La libreria standard, quindi, svolge un doppio ruolo: semplificare l'uso dei servizi del kernel e offrire **strumenti indipendenti** dal sistema operativo.

Dal punto di vista della **portabilità**, le system call sono regolate da **standard** come **POSIX**, definito dall'**IEEE**, e dallo **standard ISO C**, che assicurano la compatibilità del codice su vari sistemi UNIX e Linux conformi. Inoltre, per astrarre dalle caratteristiche dell'hardware, si ricorre a **tipi di dato definiti** tramite **typedef** all'interno di header come `<sys/types.h>`. Tali tipi, tra cui **pid_t** e **size_t**, terminano in **_t** e garantiscono un livello di astrazione che facilita lo sviluppo di codice **portabile e indipendente dall'architettura**.

Gestione dei file

La **gestione dei file** in un sistema operativo come UNIX avviene attraverso apposite **system call**, che consentono di manipolare **file regolari**, **directory** e **file speciali**. Tra questi ultimi rientrano i **link simbolici**, i **dispositivi hardware** come terminali e stampanti, e i **meccanismi di comunicazione tra processi (IPC)** come pipe e socket.

Le principali operazioni di **input/output sui file** si realizzano tramite system call specifiche come **open()**, **read()**, **write()**, **lseek()** e **close()**. Tali chiamate costituiscono l'I/O **non bufferizzato**, poiché ogni operazione corrisponde direttamente a un'interazione col **kernel**. Una sequenza tipica prevede l'uso di **open()** per aprire un file, ottenendo un **file descriptor** (**fd**), seguito da **read()** e **write()** per la lettura e scrittura, **lseek()** per lo spostamento nel file, **close()** per la chiusura e infine **unlink()** per la rimozione del file.

I **file descriptor** sono numeri interi **non negativi** usati dal kernel per rappresentare i file aperti. Ogni file aperto viene associato a uno di questi descrittori, e la loro quantità può variare da **0 a OPEN_MAX**, un limite definito da POSIX. Storicamente, UNIX limitava i file aperti a 20 per processo, ma versioni successive hanno innalzato il limite a 63 e oltre. Nei sistemi moderni come FreeBSD, Linux, macOS e Solaris, il limite è legato solo alla **memoria disponibile** e alla **dimensione dell'intero**.

Quando un file viene aperto, il kernel restituisce un **file descriptor** al processo. Questo valore sarà usato come argomento per le system call di lettura e scrittura. Esistono delle **convenzioni standard**: il descrittore **0** rappresenta lo **standard input**, l'**1** lo **standard output** e il **2** lo **standard error**. In alternativa, per maggiore leggibilità e portabilità, si possono utilizzare le **costanti simboliche** **STDIN_FILENO**, **STDOUT_FILENO** e **STDERR_FILENO**, definite nell'header `<unistd.h>`.

Chiamata di sistema open

```
#include <fcntl.h>

int open(const char *path, int oflag, /*mode_t mode*/...);
```

La **chiamata di sistema** `open ()` permette di **aprire** o **creare** un file al percorso `path`, restituendo un **file descriptor** intero non negativo che rappresenta il file aperto, oppure **-1 in caso di errore**, impostando `errno`.

Per utilizzarla, è necessario includere l'header `<fcntl.h>`.

La funzione `open ()` accetta almeno due argomenti: il **percorso del file** (`path`) e un insieme di **flag** (`oflag`), mentre un terzo parametro opzionale, `mode`, viene specificato solo quando il file deve essere creato.

Il parametro `oflag` determina il **modo di apertura** del file e può assumere uno tra i seguenti valori:

- `O_RDONLY` (solo lettura);
- `O_WRONLY` (solo scrittura);
- `O_RDWR` (lettura e scrittura).

Oltre a questi, si possono aggiungere **flag opzionali** (concatenandoli con la `|` pipe) per modificare il comportamento dell'apertura:

- `O_APPEND` esegue un'aggiunta alla fine del file per ciascuna scrittura;
- `O_CREAT` crea il file se non esiste,
- `O_EXCL`, usato insieme a `O_CREAT`, garantisce che il file venga creato solo se non esiste già, rendendo l'operazione **atomica**. Restituisce un errore se il file esiste.

Altri **flag di controllo** includono:

- `O_TRUNC`, da usare in modalità di scrittura, lo apre ed elimina il contenuto del file (non il file);
- `O_NOCTTY`, da usare con un `path` che identifica un terminale (anche una porta seriale), non lo rende il terminale di controllo del processo, quindi eventuali segnali inviati dal terminale al processo vengono ignorati (utile per i processi in background o demoni per evitare comportamenti indesiderati);
- `O_NONBLOCK`, che se `path` è una FIFO, un file a blocchi o a caratteri, apre in maniera non bloccante, sia in lettura sia in scrittura, utile per I/O asincrono.

Il parametro opzionale `mode`, invece, viene usata quando si crea un file con `O_CREAT` e definisce i **bit di permesso** per l'accesso al file quando questo viene creato. A questi bit vengono sottratti i permessi della **umask** del processo che ha invocato `open ()` e quelli non specificati nel parametro `mode` saranno impostati a 0. I permessi sono espressi tramite costanti simboliche che seguono lo schema di **lettura (R)**, **scrittura (w)** ed **esecuzione (x)** per tre categorie: **utente**, **gruppo** e **altri**.

mode	Permesso
<code>S_IRUSR</code>	Lettura utente
<code>S_IWUSR</code>	Scrittura utente
<code>S_IXUSR</code>	Esecuzione utente
<code>S_IRGRP</code>	Lettura gruppo
<code>S_IWGRP</code>	Scrittura gruppo
<code>S_IXGRP</code>	Esecuzione gruppo
<code>S_IROTH</code>	Lettura altri
<code>S_IWOTH</code>	Scrittura altri
<code>S_IXOTH</code>	Esecuzione altri

Ad esempio:

- `S_IRUSR` rappresenta il permesso di lettura per l'utente
- `S_IWGRP` quello di scrittura per il gruppo
- `S_IXOTH` quello di esecuzione per altri utenti.

Chiamata di sistema creat

```
#include <fcntl.h>

int creat(const char *path, mode_t mode);
```

La **chiamata di sistema** `creat()` fu introdotta nei primi sistemi UNIX per consentire la **creazione di file** in un contesto in cui la primitiva `open()` non supportava la creazione diretta, accettando soltanto valori per la **modalità di accesso** (lettura, scrittura, lettura/scrittura) ma non opzioni come la creazione o il troncamento del file. `creat()` accetta un **percorso** e un **valore di modalità** (`mode`), creando un nuovo file aperto in **sola scrittura** e restituendo un **file descriptor**, oppure `-1` in caso di errore.

```
open(path, O_WRONLY | O_CREAT | O_TRUNC, mode);
```

Attualmente, `creat()` è **funzionalmente equivalente** a una chiamata `open()` con i flag `O_WRONLY | O_CREAT | O_TRUNC`, ossia apre il file in scrittura, lo **crea se non esiste** e lo **tronca** se già presente. Tuttavia, con l'introduzione di flag più sofisticati nella moderna `open()`, l'uso di `creat()` è diventato **obsoleto**, poiché limitato all'apertura in scrittura. In passato, tale limitazione comportava **procedure inefficienti**, come la creazione temporanea del file con `creat()`, seguita da `close()` e successiva riapertura in lettura.

L'evoluzione della system call `open()` ha reso possibile la creazione di file **accessibili in lettura e scrittura** in un'unica istruzione, come `open(path, O_RDWR | O_CREAT | O_TRUNC, mode)`, rendendo `creat()` **ridondante** e migliorando al tempo stesso la **leggibilità** e l'**efficienza del codice**.

Chiamata di sistema close

```
#include <unistd.h>

int close(int fildes);
```

La **chiamata di sistema** `close()` serve per **chiudere un file** precedentemente aperto e rilasciare il **file descriptor** associato. La funzione è definita nell'header `<unistd.h>` e accetta come parametro l'intero `fildes`, ovvero il **descrittore di file** restituito da `open()` o da altre chiamate simili. Se l'operazione fallisce, la funzione restituisce `-1`, impostando `errno`.

Anche se è buona pratica **chiudere esplicitamente** ogni file al termine del suo utilizzo, il sistema operativo provvede comunque a **chiudere automaticamente tutti i file** aperti da un processo quando questo **termina**. Tuttavia, l'uso esplicito di `close()` è essenziale per evitare **perdite di risorse**, come il superamento del limite di file aperti o il mancato aggiornamento di dati nei buffer.

Chiamata di sistema lseek

```
#include <unistd.h>

off_t lseek (int fildes, off_t offset, int whence);
```

La **chiamata di sistema** `lseek()` permette di **modificare o leggere la posizione corrente** all'interno di un file aperto, definita da un valore chiamato **offset**. Questo offset rappresenta il numero di **byte** dall'inizio del file e viene associato a ciascun **file descriptor**. La funzione `lseek()`, dichiarata in `<unistd.h>`, restituisce il nuovo offset o `-1` in caso di errore.

Quando un file viene aperto, il suo offset è inizialmente **impostato a zero**, a meno che non venga specificata l'opzione `O_APPEND`, che forza la scrittura sempre in coda. Il parametro `whence` della `lseek()` determina da dove viene calcolato lo spostamento dell'offset. I valori possibili sono:

- `SEEK_SET`, per fissare l'offset rispetto all'**inizio del file**
- `SEEK_CUR`, per spostarlo rispetto all'**offset corrente**
- `SEEK_END`, per posizionarsi rispetto alla **fine del file**

Una delle applicazioni tipiche di `lseek()` è la lettura dell'**offset corrente** del file, ad esempio con `lseek(fd, 0, SEEK_CUR)`. Questa tecnica è anche usata per verificare se un file è "**seekable**", cioè se supporta lo spostamento casuale. Se si tenta di eseguire `lseek()` su **pipe, FIFO o socket**, la chiamata fallisce, restituendo `-1` e impostando `errno` a `ESPIPE`.

`lseek()` è uno strumento fondamentale per il controllo della **posizione di lettura o scrittura** all'interno dei file e per gestire file **ad accesso casuale**, ma è inapplicabile su stream a **flusso continuo**.

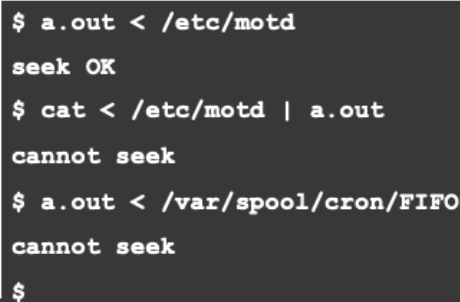
Un aspetto interessante della **chiamata di sistema `lseek()`** è che permette di **impostare l'offset oltre la fine attuale del file**, creando così una **zona vuota** nel file stesso. Quando questo avviene, la successiva `write()` **estende il file**, ma lo spazio intermedio — tra la vecchia fine e il nuovo offset — non viene realmente scritto su disco. Questo intervallo viene chiamato **buco (hole)**.

Un **buco** in un file non contiene dati effettivi e qualsiasi tentativo di lettura in quella zona restituisce **byte con valore 0**, anche se **nessun blocco fisico di memoria** è stato allocato sul disco. Questo meccanismo è supportato dai **file system sparsi (sparse files)** ed è utile per rappresentare **file molto grandi con dati parziali**, riducendo il consumo effettivo di spazio su disco.

`lseek()` non solo consente il controllo preciso dell'offset per l'accesso ai dati, ma offre anche la possibilità di gestire file **a struttura non continua**, sfruttando la **flessibilità del file system** nella gestione dello spazio.

Esempio 1

```
#include <sys/types.h>
#include "apue.h"
int main(void)
{
    if (lseek(STDIN_FILENO, 0, SEEK_CUR) == -1)
        printf("cannot seek\n");
    else
        printf("seek OK\n");
    exit(0);
}
```



```
$ a.out < /etc/motd
seek OK
$ cat < /etc/motd | a.out
cannot seek
$ a.out < /var/spool/cron/FIFO
cannot seek
$
```

Questo esempio mostra come usare la **chiamata di sistema `lseek()`** per verificare se un file (in questo caso, lo **standard input**) supporta l'operazione di **posizionamento (seek)**, cioè se è possibile **spostare l'offset corrente** nel file.

Il programma tenta di eseguire:

```
lseek(STDIN_FILENO, 0, SEEK_CUR)
```

che significa: *sposta l'offset di 0 byte rispetto alla posizione corrente* (cioè, semplicemente leggilo). Se `lseek()` restituisce `-1`, significa che il file **non è seekable**, ovvero **non supporta lo spostamento dell'offset** (tipico per pipe, FIFO, socket, o input da una pipeline). In quel caso viene stampato "cannot seek". Altrimenti, stampa "seek OK".

Esecuzioni mostrate:

- `$ a.out < /etc/motd → seek OK`
Il file `/etc/motd` è un **file regolare**, quindi supporta `lseek()`.
- `$ cat < /etc/motd | a.out → cannot seek`
In questo caso, l'input di `a.out` arriva da una **pipe**, che **non è seekable**.
- `$ a.out < /var/spool/cron/FIFO → cannot seek`
Qui l'input è una **FIFO** (una coda nominata), anch'essa **non seekable**.

Esempio 2

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include "apue.h"
char buf1[] = "abcdefghij";
char buf2[] = "ABCDEFGHIJ";
int main(void) {
    int fd;

    if ( (fd = creat("file.hole", FILE_MODE)) < 0) err_sys("creat error");

    if (write(fd, buf1, 10) != 10) err_sys("buf1 write error");
    /* offset ora = 10 */

    if (lseek(fd, 40, SEEK_SET) == -1) err_sys("lseek error");
    /* offset ora = 40 */

    if (write(fd, buf2, 10) != 10) err_sys("buf2 write error");
    /* offset ora = 50 */
    exit(0);
}
```

Questo esempio C mostra come creare un file con un **bucco (hole)** usando la chiamata di sistema `lseek()`. I **file con buchi** sono una caratteristica dei file system sparsi (sparse files) che permettono di risparmiare spazio su disco.

Analisi passo per passo:

1. Creazione del file:

```
fd = creat("file.hole", FILE_MODE);
```

Si crea un nuovo file chiamato `file.hole` in **sola scrittura**, con i permessi indicati da `FILE_MODE`.

2. Scrittura iniziale:

```
write(fd, buf1, 10);
```

Si scrive la stringa `"abcdefghij"` nel file, ovvero **10 byte**. A questo punto, l'**offset è 10**.

3. Spostamento dell'offset:

```
lseek(fd, 40, SEEK_SET);
```

L'offset viene **saltato fino al byte 40**. I **30 byte intermedi (dal 10 al 39) non vengono scritti**: questo crea un **buco** nel file. Il file system non alloca blocchi fisici per questi byte, e se letti, restituiscono **valori zero**.

4. Scrittura dopo il buco:

```
write(fd, buf2, 10);
```

Si scrive "ABCDEFGH IJ" partendo dal byte 40, quindi l'**offset finale è 50**.

Risultato:

Il file `file.hole` avrà la seguente struttura:

- Byte 0-9: "abcdefghij"
- Byte 10-39: **zero** (buco)
- Byte 40-49: "ABCDEFGH IJ"

Questo esempio dimostra come sia possibile creare **file con contenuto "discontinuo"**, dove porzioni non scritte **non occupano spazio fisico** ma vengono comunque considerate parte della dimensione logica del file. Si tratta di una tecnica utile per ottimizzare spazio e rappresentare dati sparsi.

Uso di `od` per visualizzare il contenuto

Dopo aver eseguito il programma (`./a.out`) che scrive un file con un buco (come visto in precedenza), si usa:

```
$ od -c file.hole
```

Il comando `od` (octal dump) con il flag `-c` stampa il contenuto del file come **caratteri ASCII**. I numeri a inizio riga (come 0000000 o 0040012) rappresentano l'**offset in ottale**.

L'output mostra:

```
0000000  a b c d e f g h i j \0 \0 \0 \0 \0 \0
0000020  \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0
*
0040000  A B C D E F G H I J
0040012
```

- Le prime 10 lettere "abcdefghij"
- Poi una lunga sequenza di `\0` (byte nulli), che rappresentano il **buco**
- Infine, le lettere "ABCDEFGH IJ" scritte dopo il `lseek()`

Questo conferma visivamente che nel file esiste uno spazio non scritto (il buco), riempito implicitamente da byte nulli.

Secondo esempio: confronto tra file con e senza buco

Il comando:

```
$ ls -ls file.hole file.nohole
```

mostra:

```
8  -rw-r--r--  1  staiano  50   Jan  21  10:34  file.hole
20 -rw-r--r--  1  staiano  50   Jan  21  10:38  file.nohole
```

- **file.hole** (con buco) ha una dimensione logica di 50 byte, ma **occupa solo 8 blocchi**
- **file.nohole** (senza buco), di pari dimensione logica, **occupa 20 blocchi**

Questo dimostra che i **buchi nei file non occupano spazio reale su disco**: anche se il file appare della stessa dimensione, il file system ottimizza lo spazio **non allocando blocchi** per le zone mai scritte.

Chiamata di sistema read

```
#include <unistd.h>

ssize_t read(int fd, void *buf, size_t nbytes);
```

La **chiamata di sistema read()** consente di **leggere dati da un file** aperto, a partire dalla **posizione corrente** (offset).

Prende in input un **file descriptor**, un **buffer** dove memorizzare i dati letti, e il numero di **byte da leggere**. Restituisce il numero di byte effettivamente letti, **0 se si è raggiunta la fine del file**, oppure **-1 in caso di errore**. Inoltre, aggiorna automaticamente l'**offset del file**.

In molti casi, **read()** può restituire **meno byte di quelli richiesti**. Questo comportamento è normale in diverse situazioni:

- Se si è arrivati alla **fine di un file regolare**, il sistema restituisce solo i byte rimanenti.
- Durante la lettura da un **terminale**, tipicamente viene letta **una riga alla volta**.
- Quando si legge da una **rete**, il buffering può determinare il ritorno di una quantità **inferiore** rispetto a quella richiesta.
- Se si legge da una **pipe o FIFO**, e non ci sono abbastanza dati disponibili, **read()** restituisce solo ciò che è disponibile in quel momento.
- Se l'operazione viene **interrotta da un segnale**, è possibile che **read()** restituisca una **lettura parziale**.

read() è una funzione di basso livello che offre **grande controllo**, ma richiede che il programmatore gestisca i casi in cui la lettura sia **incompleta o parziale**, specialmente in contesti asincroni o su dispositivi di input/output non regolari. Empiricamente è stato stabilito che la corretta dimensione del buffer deve essere di almeno 4096 byte.

Esempio:

```
char buffer[100];
ssize_t bytesRead = read(fd, buffer, 100);
```

Dopo la chiamata, **buffer** contiene fino a 100 byte di dati letti dal file.

bytesRead indica quanti byte sono stati effettivamente scritti in **buffer**.

Se **read()** restituisce 0, significa che sei arrivato alla **fine del file**. Se restituisce -1, c'è stato un **errore** (e **errno** spiega il motivo).

Chiamata di sistema write

```
#include <unistd.h>

ssize_t write(int fildes, const void *buf, size_t nbytes);
```

La **chiamata di sistema write ()** serve per **scrivere dati su un file** o un altro tipo di file descriptor (come socket, pipe, terminali), partendo dalla **posizione corrente** nel file.

Prende tre argomenti:

- un **file descriptor** (`fildes`) precedentemente ottenuto con `open ()`,
- un **buffer di dati** (`buf`) da cui leggere le informazioni da scrivere nel file,
- e il **numero di byte** da scrivere (`nbytes`).

La funzione **scrive i dati dal buffer nel file**, aggiornando automaticamente l'**offset** del file di conseguenza.

Restituisce il **numero di byte effettivamente scritti**, che può essere:

- **inferiore a nbytes** (in alcuni casi, ad esempio per dispositivi o pipe),
- **zero** (caso raro, ma possibile),
- oppure **-1 in caso di errore**, con `errno` che indica il tipo di errore.

Esempio (copia un file)

```
#define BUFFS 4096

int main(void)
{
    int n;
    char buf[BUFFS];
    while ((n=read(STDIN_FILENO, buf, BUFFS)) > 0)
        if (write(STDOUT_FILENO, buf, n) != n)
            printf("write error");
    if (n < 0)
        printf("read error");
    exit(0);
}
```

Come scegliamo il valore di BUFFSIZE?

Questo esempio implementa un programma che **copia l'input standard (stdin) sull'output standard (stdout)** utilizzando le chiamate di sistema **read ()** e **write ()**. All'inizio, viene definito un buffer di dimensione **4096 byte** tramite la direttiva `#define`, valore che rappresenta una **dimensione tipica di pagina di memoria** nei sistemi UNIX/Linux, scelta ottimale per ridurre le chiamate al kernel mantenendo un uso efficiente della memoria.

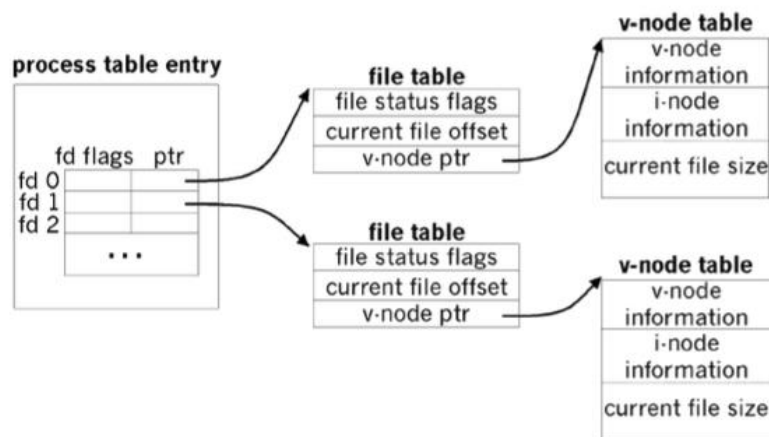
Nel corpo del programma, viene dichiarato un **buffer** e un **intero n** per memorizzare la quantità di byte letti. Il programma entra in un ciclo `while` in cui legge fino a 4096 byte alla volta da **STDIN_FILENO**, ovvero dallo standard input. Se la lettura ha successo e restituisce un valore positivo, questo valore viene utilizzato per scrivere lo stesso numero di byte su **STDOUT_FILENO**, cioè sullo standard output. Se `write ()` non riesce a scrivere tutti i byte richiesti, viene stampato un messaggio di **errore di scrittura**. Alla fine del ciclo, se `read ()` restituisce un valore negativo, si segnala un **errore di lettura**.

Questo programma funziona come una semplice **utility di copia**, simile al comportamento base del comando `cat`, ed è utilizzabile sia con input/output da file che in **pipeline**, permettendo di trattare flussi di dati in modo diretto ed efficiente. La domanda posta in fondo allo slide – “**Come scegliamo il valore di BUFSIZE?**” – invita a riflettere sull’impatto delle dimensioni del buffer: troppo piccole comportano **overhead di sistema**, troppo grandi causano **spreco di memoria**. Il valore di 4096 rappresenta dunque un compromesso efficace per garantire buone **prestazioni**.

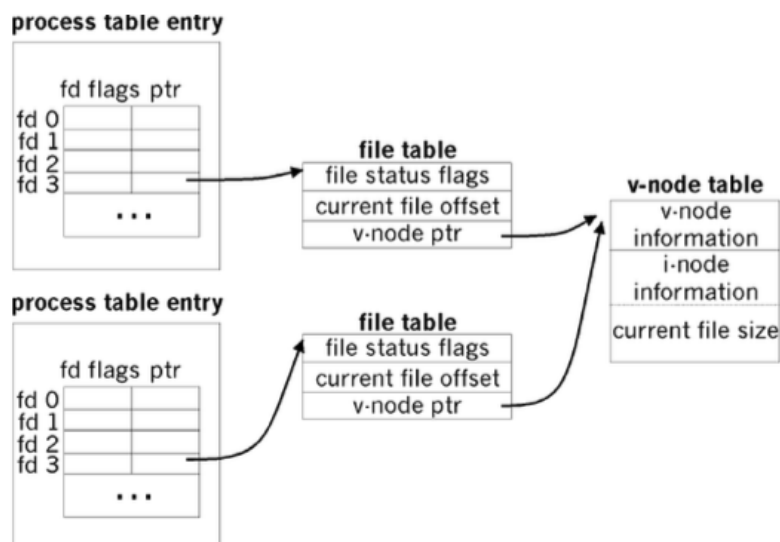
Condivisione di file

Il sistema operativo **Unix** supporta la **condivisione dei file aperti tra processi differenti**, sfruttando tre distinte **strutture dati** mantenute dal **kernel**. La relazione tra queste strutture determina l’effetto che un processo può avere su un altro quando entrambi condividono l’accesso allo stesso file.

Ogni processo ha una propria **voce nella tabella dei processi**, che include una **tabella dei descrittori di file**. Ogni descrittore è associato a dei **flag** e a un **puntatore** a una voce della tabella dei file. Il kernel mantiene anche una **tabella globale dei file aperti**, in cui ogni voce contiene i **flag dello stato del file** (ad esempio `read`, `write`, `append`), l’**offset corrente** e un **puntatore** a un elemento della tabella dei **v-node** (o **i-node** nel caso di Linux). Infine, la struttura **v-node** descrive le **informazioni del file stesso** e punta al corrispondente **i-node** del file system, contenente metadati e funzioni operative. Su Linux, non esiste il **v-node** vero e proprio, ma una struttura equivalente chiamata **i-node generico**, che serve lo stesso scopo concettuale.



Un esempio tipico è un processo con due file aperti: uno associato allo **standard input** (descrittore 0) e l’altro allo **standard output** (descrittore 1). Ciascun descrittore nella tabella dei processi punta a un elemento nella tabella dei file, e ogni voce in quest’ultima ha un puntatore al **v-node**, dove risiedono i dettagli del file nel file system.



Quando si hanno **due processi indipendenti** che aprono lo **stesso file**, ciascuno ha una propria voce nella **tabella dei descrittori** e nella **tabella dei file**, ma entrambi condividono l'accesso alla **stessa voce della tabella dei v-node**. Questo consente di accedere allo stesso file fisico, ma ogni processo mantiene il **proprio offset corrente**, permettendo un accesso indipendente.

Quando un processo accede al file con una chiamata come `write()`, l'**offset nella sua voce della tabella dei file** viene aggiornato. Se il file è stato aperto con il flag `O_APPEND`, questo viene registrato nella voce della tabella dei file, e le scritture verranno sempre posizionate in coda. Una chiamata a `lseek()` modifica solo l'offset locale del file, senza provocare I/O. Se si richiede un posizionamento alla **fine del file**, il valore viene letto dalla struttura **i-node**, che contiene le informazioni sulla **dimensione del file**.

Il kernel mantiene una separazione chiara tra il concetto di file come **entità condivisa** (tramite il v-node) e il **modo in cui ciascun processo interagisce** con il file (tramite offset e flag locali), permettendo una gestione **flessibile e controllata della condivisione**.

Operazioni atomiche

- Esempio: come aggiungere 100 byte alla fine di un file? (prime versioni di Unix)

```
if (lseek(fd, 0L, 2) < 0) /* posizionamento fine file */
    err_sys ("lseek error");
if (write(fd, buff, 100) != 100) /* scrittura */
    err_sys ("write error");
```

In questo esempio si analizza un problema legato all'**atomicità delle operazioni sui file**. Si vuole aggiungere **100 byte alla fine di un file**, come avveniva nelle prime versioni di Unix. La tecnica mostrata prevede prima un **posizionamento con `lseek()`** alla fine del file e poi una **scrittura con `write()`**. Per un **singolo processo**, questo approccio non crea problemi, ma quando più processi cercano di eseguire contemporaneamente questa sequenza, la situazione cambia drasticamente.

Il problema emerge nel momento in cui due **processi concorrenti**, come A e B, **aprono lo stesso file senza il flag `O_APPEND`**. Entrambi eseguono `lseek()` e calcolano la **stessa posizione finale** (es. offset 1500), ma poi entrano in concorrenza sulla scrittura. Se il processo B scrive per primo, e subito dopo il processo A esegue la sua scrittura partendo dalla stessa posizione, il risultato è che il **contenuto scritto da B viene sovrascritto da A**. Il file finisce quindi per contenere **solo i dati di A**, come se B non avesse mai scritto nulla.

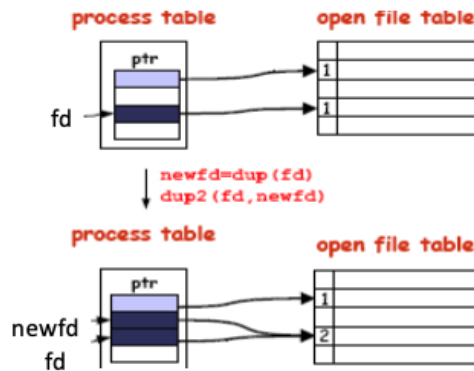
Il motivo di questo comportamento risiede nel fatto che l'operazione "posizionati alla fine del file e scrivi" non è **atomica**: si compone di **due chiamate distinte**, quindi può essere **interrotta o intercalata** da un altro processo. Ogni operazione che prevede più di una chiamata di sistema non può essere garantita come indivisibile, e ciò introduce condizioni di **race**.

Per risolvere il problema, è necessario aprire il file usando il flag `O_APPEND`. In questo modo è il **kernel** stesso che, prima di ogni `write()`, **posiziona automaticamente l'offset alla fine del file**, rendendo l'operazione **atomica**. Con `O_APPEND`, anche in presenza di più processi concorrenti, ciascuna scrittura avverrà in coda, evitando sovrascritture.

Questo esempio mostra chiaramente l'importanza dell'atomicità nelle operazioni di I/O, specialmente in **ambiente concorrente**, e come l'uso corretto dei **flag di apertura** possa prevenire comportamenti indesiderati.

Duplicare i descrittori di file

Nel sistema operativo Unix, le chiamate di sistema `dup()` e `dup2()` permettono di **duplicare i file descriptor**, creando nuovi descrittori che **puntano alla stessa entry nella tabella dei file aperti**.



In pratica, questo significa che due (o più) descrittori fanno riferimento allo stesso file, condividendo lo stesso **offset** e gli stessi **flag di stato**. Non vengono creati nuovi file o aperti nuovi stream: cambia solo il numero del descrittore.

Le system call dup e dup2

```
#include <unistd.h>

int dup(int filedes);

int dup2(int filedes, int filedes2);
```

La funzione **dup()** restituisce il **più piccolo file descriptor disponibile**, associandolo all'entry indicata. La funzione **dup2()** invece consente di **specificare il numero del nuovo file descriptor**: se questo è già in uso, viene **chiuso automaticamente** e rimpiazzato con la copia.

Entrambe le funzioni **restituiscono il nuovo descrittore**, oppure **-1 in caso di errore**. I descrittori duplicati **condividono lo stesso puntatore interno**, e quindi ogni operazione su uno influenza l'altro in termini di posizione nel file.

Esempio duplicare i descrittori di file

```
#include <stdio.h>
#include <fcntl.h>

int main (void) {
    int fd1, fd2, fd3;
    fd1 = open ("test.txt", O_CREAT | O_RDWR, 0600);
    printf ("fd1 = %d\n", fd1);
    write (fd1, "Cosa sta", 8);
    fd2 = dup (fd1); /* Effettua una copia di fd1 */
    printf ("fd2 = %d\n", fd2);
    write (fd2, " accadendo", 10);
    close (0);      /* Chiude lo standard input */
    fd3 = dup (fd1); /* Effettua un'altra copia di fd1 */
    printf ("fd3 = %d\n", fd3);
    write (0, " al contenuto", 13);
    dup2 (3, 2);    /* Duplica il canale 3 sul canale 2 */
    write (2, "?\n", 2);
    return 0;
}
```

Nell'esempio proposto, il file "test.txt" viene aperto in modalità lettura e scrittura e il relativo descrittore viene assegnato alla variabile `fd1`. In seguito, si utilizza la funzione `dup()` per duplicare `fd1`, ottenendo un secondo descrittore, `fd2`, che fa riferimento allo stesso file. Attraverso entrambi i descrittori, `fd1` e `fd2`, vengono scritti dati nel file, dimostrando che i descrittori duplicati condividono lo stesso offset di scrittura.

A un certo punto del programma, viene chiuso lo **standard input** tramite `close(0)`. Questa operazione rende il descrittore 0 nuovamente disponibile, poiché non è più associato a nessun file o dispositivo. Quando si invoca nuovamente `dup(fd1)`, il sistema operativo assegna al nuovo descrittore la prima posizione libera, che in questo caso è proprio la posizione 0. Pertanto, la variabile `fd3` assume valore 0 e rappresenta un'ulteriore copia di `fd1`, ma ora associata al descrittore che in origine era riservato allo standard input. Da quel momento, tutte le operazioni effettuate su `fd3` (o direttamente su 0) avranno effetto sul file aperto, e non più sullo standard input.

Infine, la chiamata `dup2(fd3, 2)` ha l'effetto di sostituire il descrittore 2 (standard error) con una copia di `fd3`. In questo modo, anche lo standard error viene reindirizzato al file "test.txt", e ogni successiva scrittura su `stderr` finirà all'interno dello stesso file. Questo comportamento dimostra come i descrittori di file possano essere manipolati per redirigere l'input e l'output standard verso file o dispositivi differenti, sfruttando meccanismi forniti dal sistema operativo.

```
$ ./mydup
fd1 = 3
fd2 = 4
fd3 = 0
$ cat test.txt
Cosa sta accadendo al contenuto?
$
```

Il risultato finale, come confermato da `cat test.txt`, è che il contenuto del file è frutto delle scritture concatenate di tutti i descrittori duplicati. Questo dimostra che tutti i descrittori puntano allo stesso file, con un **offset condiviso**. Ogni scrittura prosegue da dove l'ultima si era fermata, a meno di modifiche esplicite all'offset.

Gestione degli errori: `perror()` e `errno`

Per gestire correttamente gli **errori** derivanti dalle **system call**, è fondamentale comprendere l'utilizzo di due strumenti chiave: la variabile globale **errno** e la funzione di libreria **perror()**.

Ogni system call ritorna **-1** in caso di **fallimento**. Quando ciò avviene, viene aggiornato il valore di **errno**, che contiene un **codice numerico** rappresentante il tipo di errore che si è verificato.

`errno.h` contiene codici di errore predefiniti. Esempio:

```
#define EPERM 1 /* Operation not permitted */
#define ENOENT 2 /* No such file or directory */
#define ESRCH 3 /* No such process */
#define EINTR 4 /* Interrupted system call */
#define EIO 5 /* I/O error */
```

Questo valore può essere interpretato tramite l'inclusione del file **errno.h**, dove sono definiti simbolicamente i codici (es. `EPERM`, `ENOENT`, ecc.).

`void perror (char *str)`

La funzione **perror()**, che non è una system call ma una **funzione di libreria**, serve a **stampare un messaggio descrittivo** dell'errore corrente basato sul valore di **errno**. Quando viene invocata, `perror("nome")` stampa la stringa "nome: descrizione errore", rendendo più semplice il debugging. Se non vi sono errori, restituisce "Success" o "Error 0".

Esempio

```
#include <stdio.h>
#include <fcntl.h>
#include <errno.h>
int main(void) {
    int fd;
    /* Apre un file inesistente per causare un errore */
    fd = open ("nonexist.txt", O_RDONLY);
    if (fd == -1) { /* fd=-1 => si è verificato un errore */
        printf ("errno = %d\n", errno);
        perror ("main");
    }
    fd = open ("/", O_WRONLY); /* Forza un errore diverso */
    if (fd == -1) {
        printf ("errno = %d\n", errno);
        perror ("main");
    }

    /* Esegue una system call con successo */

    fd = open ("nonexist.txt", O_WRONLY | O_CREAT, 0644);

    printf("errno = %d\n", errno);
    /* Visualizza dopo la chiamata */

    perror ("main");
    errno = 0; /* Reset manuale variabile di errore */
    perror ("main");
    return 0;
}
$ showErrno
errno = 2
main: No such file or directory
errno = 21
main: Is a directory
errno = 29
main: Illegal seek
main: Success
$
```

Un esempio pratico mostra l'uso con un file inesistente e con un accesso scorretto alla root directory. Il programma apre un file con `open()`, controlla se il valore di ritorno è `-1` e in tal caso stampa **errno** e invoca **perror()** per descrivere l'errore. Inoltre, dimostra come **reset** manualmente la variabile **errno** sia utile, poiché ogni chiamata fallita può **sovrascriverne** il contenuto.

Nel terminale, eseguendo il programma `showErrno`, possiamo osservare l'output dei diversi messaggi di errore associati alle operazioni fallite, seguiti da un messaggio di **successo** quando la chiamata va a buon fine.

Esempio: copia di un file per carattere

```
#include<unistd.h>

...

int main(){
    char c;
    int in, out;
    in = open("file.in",O_RDONLY);
    out = open("file.out",O_WRONLY|O_CREAT, S_IRUSR|S_IWUSR);
    while(read(in,&c,1)==1)
        write(out,&c,1);
    exit(0);
}
```

Questo programma in C esegue la copia del contenuto di un file un carattere alla volta utilizzando le system call `open()`, `read()` e `write()`. All'inizio vengono dichiarate alcune variabili: un carattere per contenere il dato letto, e due interi che serviranno come descrittori di file. Successivamente, il file chiamato "file.in" viene aperto in modalità di sola lettura. Se l'apertura ha successo, il descrittore ottenuto viene memorizzato nella variabile `in`.

Subito dopo, viene aperto (o creato se non esiste) un file chiamato "file.out" in modalità di sola scrittura. Il file viene creato con i permessi di lettura e scrittura per l'utente. Il descrittore di questo file viene assegnato alla variabile `out`.

Il cuore del programma è un ciclo che legge un carattere per volta dal file di input e lo scrive nel file di output. La lettura continua finché `read()` restituisce 1, segnalando che è stato letto un byte. In caso contrario, quando si raggiunge la fine del file o si verifica un errore, il ciclo termina. Infine, il programma si chiude con `exit(0)`.

Nel complesso, questo codice dimostra in modo essenziale come leggere e scrivere file a basso livello in ambiente Unix, mostrando il comportamento delle system call su singoli byte. Anche se funzionale, risulta inefficiente per file di grandi dimensioni proprio perché opera un byte alla volta.

Esempio: copia di un file

```
/* mycopy src trg: crea una copia del file src e la chiama trg */
#include <stdio.h>
#include <sys/file.h>
#define BUFFSIZE 8192
int main(int argc, char *argv[]) {
    int fdSource; /* file descriptor per il file origine */
    int fdTarget; /* file descriptor per il file copia*/
    int n;
    char buf[BUFFSIZE]; /* buffer di transizione */
    fdSource = open(argv[1], O_RDONLY);
    fdTarget = open(argv[2], O_WRONLY | O_CREAT, 0600);
    /* copia il file sorgente sul target a blocchi di BUFFSIZE byte */
    while ( (n = read(fdSource, buf, BUFFSIZE)) > 0)
        if (write(fdTarget, buf, n) != n) {
            perror("write error");
            exit(1);
        }
}
```

Questo esempio mostra un semplice programma in C che realizza una **copia di un file**, copiando il contenuto da un file sorgente a uno di destinazione utilizzando blocchi di dati.

Il programma inizia includendo le librerie standard necessarie e definendo una **costante** chiamata `BUFSIZE` pari a 8192, che rappresenta la dimensione del buffer utilizzato per leggere e scrivere i dati.

All'interno della funzione `main`, si definiscono due **descrittori di file**, uno per il file sorgente (`fdSource`) e uno per il file destinazione (`fdTarget`), insieme a una variabile intera `n` e a un **buffer di transizione** `buf` che contiene i dati letti.

Il file sorgente viene aperto in sola lettura, mentre quello di destinazione viene aperto in sola scrittura, e se non esiste viene creato con i **permessi 0600** (lettura e scrittura per l'utente).

La parte centrale del programma è un ciclo `while` che legge blocchi di massimo `BUFSIZE` byte dal file sorgente e li scrive sul file di destinazione. Se la scrittura non ha successo (ovvero se non viene scritto esattamente il numero di byte letti), viene segnalato un errore tramite `perror("write error")`.

Il programma si chiude con `exit(1)` in caso di errore, altrimenti termina normalmente. Questo approccio è molto più efficiente rispetto alla copia un byte alla volta, specialmente per file di grandi dimensioni.

Esempio: stampa inversa di un file

```
#include <sys/file.h>

int main(int argc, char *argv[]) {
    int fd;
    char buff;
    const int charSize = sizeof(char);
    if (argc != 2) {
        printf("Utilizzo: invert <txtFile>\n");
        exit(1);
    }
    fd = open(argv[1], O_RDONLY);

    /* Si posiziona alla fine del file + 1 */
    lseek(fd, 1, SEEK_END);

    /* legge il file al contrario: ad ogni passo sposta il puntatore di due passi indietro, perché la lettura lo
    aggiorna */
    while (lseek(fd, -2*charSize, SEEK_CUR) != -1) {
        read(fd, &buff, charSize);
        printf("%c", buff);
    }
}
```

Questo programma in C serve a **stampare il contenuto di un file al contrario**, partendo dalla fine e procedendo a ritroso fino all'inizio.

All'inizio vengono dichiarate le librerie necessarie e definite le variabili: un **descrittore di file** `fd`, un **buffer** `buff` per memorizzare un carattere alla volta e una costante `charSize` che rappresenta la dimensione di un carattere.

Il programma controlla se è stato passato esattamente un argomento da linea di comando (cioè il nome del file da leggere). Se l'utente non fornisce il nome del file, viene stampato un **messaggio di utilizzo** e il programma termina.

Il file viene quindi aperto in modalità **sola lettura**. Una volta aperto, il puntatore del file viene posizionato **alla fine del file** più un byte, con la chiamata `lseek(fd, 1, SEEK_END)`. Questo posizionamento è necessario per poter effettuare il ciclo di lettura a ritroso.

Il ciclo `while` successivo è il cuore del programma. A ogni iterazione, la funzione `lseek` sposta il puntatore del file **di due caratteri indietro** (uno per annullare l'avanzamento causato dalla lettura e uno per retrocedere). Dopo aver riposizionato il puntatore, viene letto un singolo carattere con `read` e stampato a schermo con `printf`.

In questo modo, il contenuto del file viene letto **carattere per carattere in ordine inverso**, permettendo di visualizzare l'intero contenuto del file **dalla fine all'inizio**.

Capitolo 6 – File e Directory

In Unix, la maggior parte dei file rientra in due grandi categorie: **file regolari** e **directory**, ma esistono anche altri tipi di file:

- I **file regolari** sono quelli più comuni e contengono dati in qualche forma, che siano di testo o binari; per il **kernel** non c'è differenza.
- Le **directory** contengono nomi di altri file e riferimenti alle informazioni su di essi.
- **File speciali a blocchi**, utilizzati per rappresentare dispositivi a blocchi come i dischi,
- **File speciali a caratteri**, usati per dispositivi che lavorano con flussi di caratteri, ad esempio terminali e stampanti.
- File **FIFO**, utili per la comunicazione tra processi.
- **Socket**, impiegati per la comunicazione in rete.
- **Link simbolici**, che puntano ad altri file nel filesystem.

System call stat, fstat e lstat

```
#include <sys/stat.h>

int stat (const char *path, struct stat *buf);
int fstat (int filedes, struct stat *buf);
int lstat (const char *path, struct stat *buf);
```

Per ottenere informazioni su un file, Unix mette a disposizione le system call **stat**, **fstat** e **lstat**, definite nell'header `<sys/stat.h>`.

- La funzione **stat** permette di ottenere informazioni su un file specificando il **percorso**.
- La **fstat**, invece, agisce sul file già aperto, identificato tramite un **file descriptor**.
- La **lstat** restituisce informazioni sul **link simbolico** e non sul file effettivamente referenziato. Tutte queste funzioni hanno in comune un secondo argomento: un puntatore a una **struttura stat**, che verrà riempita con i dati del file.

Non è necessario avere permessi di lettura sul file per poter accedere a queste informazioni. Basta avere i permessi di **ricerca** sulle directory che compongono il **path**.

La struttura stat

```
struct stat
{
    dev_t      st_dev;      /* device number (file system) */
    ino_t      st_ino;      /* inode */
    mode_t     st_mode;     /* file type & protection */
    nlink_t    st_nlink;    /* number of hard links */
    uid_t      st_uid;      /* user ID of owner */
    gid_t      st_gid;      /* group ID of owner */
    dev_t      st_rdev;     /* device number for special files */
    off_t      st_size;     /* total size, in bytes */
    unsigned long st_blksize; /* blocksize for filesystem I/O */
    unsigned long st_blocks; /* number of blocks allocated */
    time_t     st_atime;    /* time of last access */
    time_t     st_mtime;    /* time of last modification */
    time_t     st_ctime;    /* time of last status change */
};
```

La struttura `stat` contiene molti campi. Alcuni esempi:

- **st_dev** indica il numero del dispositivo,
- **st_ino** l'inode del file,
- **st_mode** descrive tipo di file e permessi,
- **st_nlink** il numero di hard link,
- **st_uid** e **st_gid** gli identificatori del proprietario e del gruppo,
- **st_size** la dimensione del file in byte,
- **st_atime**, **st_mtime** e **st_ctime** indicano rispettivamente l'ultimo accesso, l'ultima modifica e l'ultimo cambiamento dello stato.

Tipi di file

Per determinare il tipo di file, si analizza il campo **st_mode** con l'ausilio di alcune macro definite sempre in `<sys/stat.h>`. Tra queste ci sono:

- **S_ISLNK()** per link simbolici,
- **S_ISREG()** per file regolari,
- **S_ISDIR()** per directory,
- **S_ISCHR()** per dispositivi a caratteri,
- **S_ISBLK()** per dispositivi a blocchi,
- **S_ISFIFO()** per FIFO
- **S_ISSOCK()** per socket.

L'argomento di ciascuna macro è il campo **st_mode**: es. `S_ISREG(sb.st_mode)`.

Esempio

```
#include <sys/types.h>
#include <sys/stat.h>
int main(int argc, char *argv[])
{
    int            i;
    struct stat     buf;
    char           *ptr;
    for (i = 1; i < argc; i++) {
        printf("%s: ", argv[i]);
        if (lstat(argv[i], &buf) < 0) {
            printf("lstat error\n");
            continue;
        }
        if (S_ISREG(buf.st_mode))      ptr = "regular";
        ...
        else if (S_ISBLK(buf.st_mode)) ptr = "block special";

        else      ptr = "*** unknown mode ***";
        printf("%s\n", ptr);
    }
    exit(0);
}
```

Un programma può utilizzare queste macro per stampare il tipo di ciascun file passato come argomento, e a seconda del risultato della chiamata **lstat()**, si associa una stringa che ne descrive il tipo.

```
$ ./a.out /etc/passwd /etc /dev/initctl /dev/log /dev/tty \
> /dev/scsi/host0/bus0/target0/lun0/cd /dev/cdrom
/etc/passwd: regular
/etc: directory
/dev/initctl: fifo
/dev/log: socket
/dev/tty: character special
/dev/scsi/host0/bus0/target0/lun0/cd: block special
/dev/cdrom: symbolic link
```

L'esecuzione di tale programma mostra, ad esempio, che `/etc/passwd` è un file regolare, `/etc` è una directory, `/dev/tty` è un dispositivo a caratteri, mentre `/dev/cdrom` è un link simbolico.

User ID e Group ID

Nei sistemi Unix, ogni file è associato a un proprietario e a un gruppo. Tali informazioni sono registrate nella struttura dati `stat`, all'interno dei campi `st_uid` (user ID) e `st_gid` (group ID). Questi identificatori permettono al sistema operativo di stabilire chi può accedere al file e con quali privilegi.

```
struct stat
{
    dev_t      st_dev;          /* device number (file system) */
    ino_t      st_ino;          /* inode */
    mode_t     st_mode;         /* file type & protection */
    nlink_t    st_nlink;       /* number of hard links */
    uid_t      st_uid;          /* user ID of owner */
    gid_t      st_gid;          /* group ID of owner */
    dev_t      st_rdev;         /* device number for special files */
    off_t      st_size;         /* total size, in bytes */
    unsigned long st_blksize;   /* blocksizes for filesystem I/O */
    unsigned long st_blocks;    /* number of blocks allocated */
    time_t     st_atime;        /* time of last access */
    time_t     st_mtime;        /* time of last modification */
    time_t     st_ctime;        /* time of last status change */
};
```

Set User ID e Set Group ID

Durante l'esecuzione di un processo, il sistema operativo associa una serie di **identificatori di utente e di gruppo** che determinano sia l'identità del processo, sia i suoi **diritti di accesso** alle risorse del sistema. I più importanti tra questi sono il **real user ID** e il **real group ID**, che rappresentano l'utente e il gruppo che hanno **avviato** il processo. Parallelamente, vengono assegnati un **effective user ID** e un **effective group ID**, utilizzati dal **kernel** per valutare i **permessi di accesso ai file** e alle risorse. Inoltre, un processo può appartenere anche a uno o più **supplementary group ID**, ovvero gruppi aggiuntivi a cui l'utente è associato.

In condizioni normali, gli identificatori **effettivi** coincidono con quelli **reali**, garantendo che il processo operi con i privilegi dell'utente che lo ha avviato. Tuttavia, esistono **meccanismi speciali** che consentono di **modificare temporaneamente** questi identificatori, permettendo al processo di acquisire privilegi **superiori**, necessari per eseguire operazioni altrimenti vietate. Tali meccanismi sono rappresentati dai bit **set-user-ID (SUID)** e **set-group-ID (SGID)**, che agiscono rispettivamente sull'UID e sul GID del processo.

Set user-ID

Il meccanismo del **set-user-ID** consente di eseguire un programma con i privilegi dell'**utente proprietario del file**, anziché con quelli dell'utente che lo avvia. In un contesto standard, un programma viene eseguito con i permessi dell'utente che lo lancia, ma attivando il bit **SUID** nel campo `st_mode` della struttura `stat`, si stabilisce che, all'avvio del programma, l'**effective UID** del processo sia quello del **proprietario del file eseguibile**.

Questo comportamento è fondamentale per quei programmi che devono accedere a **risorse protette**. Un esempio classico è il comando **passwd**, che consente a un utente ordinario di modificare la propria password. Il file `/etc/shadow`, che contiene le password in forma cifrata, è accessibile in scrittura solo a **root**. Per garantire che anche un utente normale possa aggiornare la propria voce in sicurezza, il file eseguibile `/usr/bin/passwd` è di proprietà di **root** e ha il bit **SUID** attivo. In questo modo, quando viene eseguito, il processo agisce con **EUID = 0**, consentendo l'operazione privilegiata pur mantenendo il controllo sull'uso.

Nel sistema dei permessi, un file eseguibile con il bit SUID appare con una **s** al posto della **x** nei permessi dell'utente: ad esempio, `-rwsr-xr-x` indica che il programma è eseguibile e `setuid` è attivo. La presenza della **S maiuscola**, come in `-rwsSr--`, indica invece che il bit SUID è attivo ma il file **non è eseguibile**, rendendo l'impostazione inutile o potenzialmente errata.

È importante sottolineare che **l'utente che esegue il programma non eredita i privilegi globali del proprietario del file**. Solo il **processo** eseguito acquisisce temporaneamente quell'identità, e **solo per le operazioni previste dal codice**. Se il programma è scritto in modo insicuro, ad esempio utilizzando chiamate a `system()` con input non controllato, un utente può sfruttarlo per **eseguire comandi arbitrari** con i privilegi del proprietario del file, aprendo la strada a una **escalation di privilegi**.

Set group-ID

Il meccanismo del **set-group-ID** funziona in modo analogo al SUID, ma riguarda il **group ID** anziché lo user ID. Attivando il bit SGID, identificato nella struttura `stat` dalla macro `S_ISGID`, il processo viene eseguito con l'**effective GID del file eseguibile**, e non con quello dell'utente che lo ha avviato. Questo è utile in scenari in cui l'appartenenza a un gruppo specifico è necessaria per accedere a file o directory protette da permessi di gruppo.

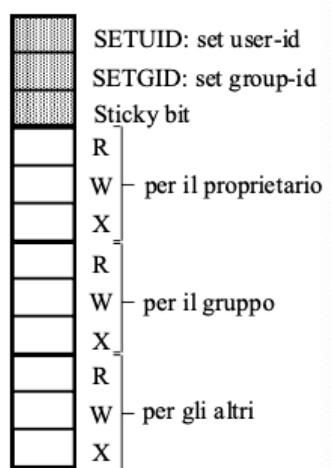
Oltre che sui file eseguibili, il bit SGID ha un comportamento molto utile quando viene applicato a una **directory**. In tal caso, tutti i file creati all'interno della directory **ereditano il GID della directory**, anziché quello del processo che li ha generati. Questo comportamento è utilizzato nei **progetti collaborativi**, dove si vuole che tutti i file prodotti da membri diversi del team appartengano automaticamente allo stesso **gruppo di lavoro**.

Nel sistema dei permessi, il bit SGID è visibile come una `s` nel campo dei permessi del gruppo, ad esempio: `rwxr-sr-x`. Se invece la directory non è eseguibile, ma il bit è impostato, apparirà una `S` maiuscola, indicando che il bit è attivo ma la directory non può essere attraversata, rendendo l'effetto nullo.

Assieme a SUID e SGID, esiste anche il bit **sticky (S_ISVTX)**, che è utile soprattutto nelle directory pubbliche come `/tmp`, dove più utenti possono scrivere, ma **solo il proprietario di un file può eliminarlo**. Il valore simbolico dei bit speciali è: `S_ISUID = 4000`, `S_ISGID = 2000`, `S_ISVTX = 1000`.

<code>S_ISUID</code>	<code>4000</code>	set UID bit
<code>S_ISGID</code>	<code>2000</code>	set GID bit
<code>S_ISVTX</code>	<code>1000</code>	sticky bit

Permessi di accesso ai file



Nel sistema operativo Unix, l'accesso ai file e alle directory è regolato da un sistema di **permessi** che consente un controllo preciso e sicuro sull'utilizzo delle risorse da parte di utenti e processi. I permessi sono rappresentati da un insieme di **12 bit**, contenuti nel campo `st_mode` della struttura `stat`. Questi bit definiscono non solo i classici permessi di **lettura (read)**, **scrittura (write)** ed **esecuzione (execute)**, ma includono anche **tre bit speciali**: il **set-user-ID (SUID)**, il **set-group-ID (SGID)** e lo **sticky bit**.

I permessi ordinari sono organizzati in tre gruppi, corrispondenti a tre categorie di utenti: il **proprietario** del file, il **gruppo** a cui il file appartiene, e **gli altri utenti**. Per ciascuna categoria vengono assegnati tre bit: uno per la lettura, uno per la scrittura e uno

per l'esecuzione. Tali permessi sono rappresentati simbolicamente e manipolabili attraverso costanti definite nella libreria `<sys/stat.h>`, come ad esempio `S_IRUSR` (lettura per il proprietario), `S_IWGRP` (scrittura per il gruppo) o `S_IXOTH` (esecuzione per gli altri). I valori sono espressi in formato ottale e possono essere combinati per configurare i diritti di accesso desiderati.

Il campo `st_mode` contiene anche informazioni sul **tipo di file**, distinguendo tra file regolari, directory, dispositivi speciali, link simbolici, e così via. Tuttavia, ai fini del controllo degli accessi, ciò che interessa principalmente sono i bit di **permesso**, che il kernel interpreta per decidere se un'operazione richiesta da un processo è autorizzata.

<code>S_IRUSR</code>	<code>0400</code>	owner read
<code>S_IWUSR</code>	<code>0200</code>	owner write
<code>S_IXUSR</code>	<code>0100</code>	owner execute
<code>S_IRGRP</code>	<code>0040</code>	group read
<code>S_IWGRP</code>	<code>0020</code>	group write
<code>S_IXGRP</code>	<code>0010</code>	group execute
<code>S_IROTH</code>	<code>0004</code>	others read
<code>S_IWOTH</code>	<code>0002</code>	others write
<code>S_IXOTH</code>	<code>0001</code>	others execute

Controllo dei permessi da parte del kernel

Ogni volta che un processo tenta di aprire, modificare o eliminare un file, il **kernel** del sistema Unix effettua una verifica dei permessi, basandosi sull'identità del file (campo `st_uid` e `st_gid`) e sugli identificatori del processo, in particolare l'**effective user ID**, l'**effective group ID**, e gli eventuali **supplementary group ID**.

Il controllo dei permessi segue una sequenza ben definita:

- Se l'**EUID** del processo è **0**, ovvero se il processo è eseguito da **root**, l'accesso è concesso automaticamente, salvo rare eccezioni.
- Se non è root, il kernel verifica se l'**EUID** del processo coincide con lo **UID** del file. Se sì, controlla i permessi per il **proprietario**.
- In assenza di corrispondenza tra UID, controlla se l'**EGID** o uno dei **supplementary group ID** coincide con il GID del file. Se sì, valuta i permessi per il **gruppo**.
- Se nessuna delle condizioni precedenti è soddisfatta, vengono infine applicati i permessi definiti per **altri utenti**.

Questo schema garantisce che le operazioni su file e directory siano autorizzate solo in presenza dei corretti privilegi, assicurando l'**isolamento tra utenti** e la **protezione delle risorse**.

Significato dei permessi in base al tipo di oggetto

I permessi hanno effetti diversi a seconda che si tratti di un **file regolare** o di una **directory**. Nel caso dei file, il permesso di **lettura** permette di aprire e leggere il contenuto, il permesso di **scrittura** consente di modificare il contenuto, e quello di **esecuzione** abilita l'avvio del file come programma. Per eseguire un file, oltre al permesso `x`, il file deve essere di tipo regolare.

Per le **directory**:

- il permesso di **lettura** permette di elencarne il contenuto;
- il permesso di **esecuzione** (chiamato anche *permesso di ricerca*) è necessario per accedere ai file contenuti o per attraversare la directory;
- il permesso di **scrittura**, invece, permette di creare o eliminare file nella directory, ma per cancellare un file **non serve avere permessi sul file stesso**: è sufficiente avere permessi di **scrittura ed esecuzione sulla directory** che lo contiene.

Inoltre, per accedere a un file tramite il suo percorso assoluto, è necessario disporre del permesso di **esecuzione su tutte le directory intermedie**, comprese quelle superiori.

Sticky bit

Il **sticky bit**, rappresentato dalla costante `S_ISVTX`, ha assunto nel tempo un significato diverso da quello originario. Oggi viene usato quasi esclusivamente su **directory condivise** come `/tmp`. Quando è attivo, impedisce a un utente con permessi di scrittura sulla directory di **rinominare o cancellare file** che non gli appartengono, a meno che non sia il **proprietario del file**, della **directory**, oppure **root**. Questo garantisce che ogni utente possa creare file in una directory pubblica senza rischiare che vengano modificati o eliminati da altri.

Meccanismi avanzati di identificazione: saved UID e GID

Oltre agli identificatori reali ed effettivi, Unix gestisce anche i cosiddetti **saved set-user-ID** e **saved set-group-ID**, che conservano una copia temporanea degli **effective ID**. Questi valori sono utilizzati per **ripristinare** i privilegi temporaneamente sospesi e sono particolarmente importanti nei programmi che devono **attivare e disattivare privilegi speciali** in modo controllato, come accade nei programmi con bit `setuid`.

System call access

```
#include<unistd.h>
int access (const char *pathname, int mode)
```

Nel sistema Unix, la system call **access ()** consente a un processo di **verificare in anticipo** se possiede i **permessi necessari per accedere a un file**, sulla base dell'identità associata al **real user ID** e al **real group ID** del processo, e **non** in base agli **effective ID**. Questo comportamento è particolarmente utile per sapere, ad esempio, se un processo potrà aprire con successo un file prima di tentare effettivamente l'operazione.

La funzione **access ()** accetta due parametri. Il primo, chiamato **pathname**, rappresenta il **percorso del file** da controllare. Il secondo parametro, denominato **mode**, specifica il **tipo di verifica** da effettuare. È possibile richiedere semplicemente la verifica dell'**esistenza del file** utilizzando il valore **F_OK**, oppure controllare se si hanno i **permessi di lettura (R_OK)**, **scrittura (W_OK)** o **esecuzione (X_OK)**. Queste modalità possono anche essere **combinare** tra loro mediante l'operatore **OR (|)**, rendendo possibile un controllo multiplo in un'unica chiamata.

Il risultato della funzione fornisce un'indicazione chiara: se l'**accesso è consentito**, viene restituito **0**; in caso contrario, il valore restituito è **-1**, ad indicare un **fallimento** della verifica.

Un aspetto interessante emerge quando si confronta il comportamento di `access ()` con quello di altre system call come `open ()`, che invece **valutano i permessi sulla base degli effective ID** del processo. In questi casi, può verificarsi una **discrepanza** tra il risultato della verifica preventiva e quello effettivo al momento dell'apertura del file.

Un esempio emblematico si ha quando un **file binario è eseguito con il flag set-user-ID attivo**, impostato ad esempio tramite il comando `chmod u+s`. In tale situazione, anche se l'utente reale **non ha i permessi di lettura** su un file riservato, come ad esempio **/etc/shadow**, l'esecuzione del programma avviene con i **privilegi dell'utente root**, grazie al meccanismo del **set-user-ID**. In questo modo, l'operazione di apertura del file riesce, anche se `access ()` avrebbe indicato il contrario. Questo caso dimostra chiaramente la **differenza tra un controllo simulato**, basato sull'identità reale, e un **controllo effettivo**, che dipende dai permessi attivi del processo in esecuzione.

Esempio (Analisi del comportamento delle system call `access ()` e `open ()` in presenza del bit Set-UID)

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>
#include <string.h>
```



```

int main() {
    const char *filename = "/etc/shadow";

    // Verifica con access() - usa real UID
    if (access(filename, R_OK) == 0) {
        printf("access(): Permessi di lettura SUCCESSFUL per %s\n", filename);
    } else {
        printf("access(): Permessi di lettura NEGATO per %s\n", filename);
    }

    // Tentativo di apertura del file - usa effective UID
    int fd = open(filename, O_RDONLY);
    if (fd >= 0) {
        printf("open(): Apertura del file riuscita!\n");
        close(fd);
    } else {
        printf("open(): Apertura del file fallita: %s\n", strerror(errno));
    }

    return 0;
}

```

L'esperimento ha avuto come obiettivo l'analisi del comportamento delle system call `access()` e `open()` nel contesto della gestione dei permessi di accesso ai file, con particolare attenzione alla distinzione tra identificatori reali ed effettivi del processo. A tal fine, è stato realizzato un programma in linguaggio C denominato `check_shadow`, progettato per verificare la possibilità di lettura del file `/etc/shadow`, il quale contiene informazioni sensibili ed è accessibile esclusivamente all'utente **root**.

Il programma in oggetto effettua una doppia verifica: dapprima mediante la chiamata `access()`, che controlla i permessi sulla base del **real user ID** e del **real group ID** del processo, e successivamente mediante la chiamata `open()`, che valuta i permessi secondo l'**effective user ID**. Questa differenza è fondamentale per comprendere come il sistema operativo Unix distingua tra **verifica preliminare** dei permessi e **verifica effettiva** al momento dell'accesso.

Per mettere in evidenza tale distinzione, è stato necessario configurare il file eseguibile `check_shadow` in modo che venisse eseguito con i privilegi dell'utente **root**. A tal scopo, si è proceduto con l'assegnazione della proprietà del file a **root**, tramite il comando `chown root:root check_shadow`, e con l'attivazione del **bit Set-UID** mediante `chmod u+s check_shadow`. In questo modo, alla sua esecuzione da parte di un utente non privilegiato, il processo risultante mantiene il real user ID dell'utente chiamante, ma assume come effective user ID quello dell'utente **root**, ereditato dal proprietario del file eseguibile.

All'esecuzione del programma, la system call `access()` ha negato l'accesso al file `/etc/shadow`, coerentemente con i privilegi associati all'identità reale dell'utente. Tuttavia, la successiva apertura del file mediante `open()` è avvenuta con successo, grazie all'elevazione temporanea dei privilegi determinata dal bit Set-UID. Tale comportamento dimostra che il controllo dei permessi effettuato da `access()` non riflette necessariamente l'effettiva possibilità di accedere a una risorsa, qualora il processo venga eseguito con privilegi superiori.

Questo esperimento conferma quindi che `access()` rappresenta uno strumento utile per simulare un controllo preliminare, ma che non può sostituire le verifiche effettuate dal kernel durante l'esecuzione vera e propria, basate sugli identificatori effettivi. Il caso considerato evidenzia inoltre il ruolo critico del bit Set-UID nella gestione dei privilegi temporanei nei sistemi Unix-like, e ne sottolinea l'importanza nella progettazione di applicazioni sicure.

Proprietà dei nuovi file e directory

Quando un processo crea un nuovo file, l'**user ID** associato a tale file viene impostato sul **effective user ID** del processo che lo ha generato. Per quanto riguarda l'assegnazione del **group ID**, lo standard **POSIX.1** lascia libertà all'implementazione del sistema operativo, prevedendo due possibili strategie.

La prima strategia assegna al nuovo file il **effective group ID** del processo. La seconda strategia, invece, assegna il **group ID** della **directory** in cui il file viene creato. La scelta tra queste due modalità dipende dal sistema in uso: ad esempio, **FreeBSD** adotta sempre la seconda modalità, utilizzando il group ID della directory.

Nel caso di **Linux**, in particolare con filesystem come **ext2** ed **ext3**, è possibile configurare il comportamento tramite un **flag** specifico da passare alla funzione **mount**. A partire dalla versione 4.2.22 del kernel Linux, oppure su sistemi come **Solaris 9**, il comportamento può essere condizionato anche dalla presenza del **bit set-group-ID** sulla directory in cui avviene la creazione del file. In questo caso, se il bit è impostato, i file creati ereditano il **group ID** della directory stessa, altrimenti viene utilizzato l'effective group ID del processo.

Permessi e creazione dei file: system call umask

```
#include <sys/stat.h>
mode_t umask(mode_t mask);
```

La system call **umask()** è impiegata nei sistemi Unix per definire i **permessi da escludere** automaticamente durante la **creazione di nuovi file o directory** da parte di un processo. Il suo comportamento consiste nell'applicare una **maschera di bit** che modifica i permessi specificati nella chiamata di sistema utilizzata per creare il file, come **creat()** o **open()**. I bit presenti nella maschera rappresentano i **permessi che devono essere negati** e vengono sottratti ai permessi richiesti tramite un'operazione **bitwise AND** con il **complemento** della maschera stessa.

La funzione può accettare due forme di argomento. Nella pratica più comune, si utilizza un **valore ottale** che rappresenta direttamente la combinazione dei permessi da rimuovere.

- $666 - 022 = 644 = rw - r - - r - -$
 - Ovvero $110\ 110\ 110 \& \sim(000\ 010\ 010) = 110\ 100\ 100$

Ad esempio, passando il valore **022** a **umask()**, si richiede che vengano esclusi i permessi di **scrittura per il gruppo** e per **altri utenti**, ottenendo così che un file, creato con permessi teorici pari a **666** (lettura e scrittura per tutti), risulti in realtà con permessi **644** (lettura e scrittura per il proprietario, sola lettura per gruppo e altri). Questo è ottenuto poiché $666 - 022 = 644$, o in binario $110110110 \& \sim(000010010) = 110100100$.

In alternativa, è possibile fornire a **umask()** una **combinazione di costanti simboliche**, come **S_IWGRP** e **S_IROTH**, definite nelle librerie POSIX. Questa modalità risulta più descrittiva e leggibile, in quanto esplicita i permessi da inibire attraverso nomi simbolici invece che mediante valori numerici.

La system call **umask()** restituisce sempre il valore **precedente** della maschera. Questo è utile se si desidera **ripristinare** la maschera originale dopo averla temporaneamente modificata. È pratica comune salvare questo valore in una variabile, ad esempio:

```
mode_t old_umask = umask(0022);

// ...

umask(old_umask); // ripristina la maschera precedente
```

Esempio

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include "apue.h"

int main(void) {

    umask(0);
    if (creat("foo", S_IRUSR | S_IWUSR | S_IRGRP | S_IWGRP | S_IROTH | S_IWOTH) < 0)
        err_sys("creat error for foo");

    umask(S_IRGRP | S_IWGRP | S_IROTH | S_IWOTH);
    if (creat("bar", S_IRUSR | S_IWUSR | S_IRGRP | S_IWGRP | S_IROTH | S_IWOTH) < 0)
        err_sys("creat error for bar");
    exit(0);
}
```

Nell'esempio riportato, un primo file viene creato con **umask** azzerata; quindi, tutti i permessi specificati nella chiamata a `creat()` vengono mantenuti. Successivamente, viene impostata una nuova **umask** che esclude alcuni permessi (lettura e scrittura per gruppo e altri), e un secondo file, il file “bar” viene creato con permessi più restrittivi, solo lettura e scrittura per User, mentre i permessi di lettura e scrittura sono esclusi per i gruppi e others. Questo comportamento è essenziale per garantire che i file creati da un processo non siano troppo permissivi rispetto alle intenzioni del programmatore o ai requisiti di sicurezza del sistema.

System call `chmod` e `fchmod`

```
#include <sys/stat.h>

int chmod (const char *path, mode_t mode);

int fchmod (int fildes, mode_t mode);
```

Il sistema operativo fornisce le system call **chmod** e **fchmod** per modificare i **permessi di accesso** associati a un file. La funzione `chmod` opera su un file identificato dal **percorso** (path), mentre `fchmod` agisce su un file già **aperto**, identificato dal relativo **file descriptor**. Entrambe restituiscono **0** in caso di successo, oppure **-1** in caso di errore.

Per modificare i permessi di un file, è necessario che l'**effective user ID** del processo che esegue la chiamata coincida con l'**user ID del proprietario** del file. In alternativa, il processo deve disporre dei privilegi del **superutente**.

```

#include      <sys/types.h>
#include      <sys/stat.h>
#include      "apue.h"
int main(void)
{
    struct stat  statbuf;
    /* modo assoluto a "rw-r--r--" */
    if (chmod("bar", S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH) < 0)
        err_sys("chmod error for bar");
    /* imposta set-group-ID disattiva group-execute */
    if (stat("foo", &statbuf) < 0)
        err_sys("stat error for foo");
    if (chmod("foo", (statbuf.st_mode & ~S_IXGRP) | S_ISGID) < 0)
        err_sys("chmod error for foo");

    exit(0);
}

```

L'esempio proposto illustra il funzionamento delle system call `chmod()` e `stat()` in relazione alla gestione dei **permessi di accesso** e del **bit set-group-ID** nei sistemi Unix. Nella prima parte del codice, viene effettuata una chiamata a `chmod()` per impostare i permessi del file "bar" in modalità assoluta, assegnando al proprietario il diritto di **lettura e scrittura**, mentre al gruppo e agli altri utenti viene concesso il solo **permesso di lettura**. In termini simbolici, ciò corrisponde ai permessi `rw-r--r--`.

Successivamente, viene esaminato lo stato del file "foo" tramite la system call `stat()`, che consente di accedere alla struttura `stat`, all'interno della quale sono memorizzati i metadati del file, tra cui il campo `st_mode`, che rappresenta la combinazione dei permessi attuali. Sulla base di tali permessi, viene poi effettuata una nuova chiamata a `chmod()`, la quale ha lo scopo di **modificare selettivamente** i permessi esistenti: in particolare, si **rimuove il permesso di esecuzione per il gruppo** (tramite l'operazione `~S_IXGRP`) e si **attiva il bit set-group-ID** (utilizzando la costante `S_ISGID`).

Questa operazione evidenzia un comportamento specifico del sistema: qualora venga attivato il **bit SGID** su un file **non eseguibile dal gruppo**, il sistema provvede automaticamente a **disattivare il bit SGID**. Questo meccanismo è motivato da considerazioni di **sicurezza**, in quanto il **bit SGID**, se attivo su un file eseguibile, consente al processo risultante dall'esecuzione di **assumere il gruppo del file**, indipendentemente dal gruppo effettivo dell'utente che lo ha eseguito. Tuttavia, tale comportamento ha senso solo se il gruppo **può effettivamente eseguire il file**. Se ciò non fosse possibile, il mantenimento del bit SGID sarebbe incoerente e potenzialmente pericoloso, motivo per cui il sistema ne forza la rimozione.

L'esempio fornisce quindi un'illustrazione concreta della logica con cui il sistema operativo Unix **valida e applica i permessi**, e mostra come le modifiche ai permessi devono tenere conto della coerenza semantica tra i bit di esecuzione e quelli speciali, come il **set-group-ID**, nell'ottica della **protezione del sistema**.

OUTPUT

```

alex@debianbox:~/Documenti/VsCode/Lezione_6_Directory/04.SysCall_chmod()$ ls -la
totale 32
drwxr-xr-x 2 alex alex 4096 1 giu 08.39 .
drwxr-xr-x 6 alex alex 4096 1 giu 08.31 ..
-rw-r--r-- 1 alex alex 0 1 giu 08.26 bar
-rwxr-xr-x 1 alex alex 71368 1 giu 08.39 Esempio
-rw-r--r-- 1 alex alex 534 1 giu 08.37 Esempio.c
-rw-rwSr-- 1 alex alex 0 1 giu 08.26 foo

```

L'output della seconda immagine mostra che il file `bar` ha permessi `-rw-r--r--`, il che è esattamente quanto impostato dalla prima chiamata a `chmod()` nel programma C (`S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH`): il proprietario ha lettura e scrittura, mentre gruppo e altri hanno solo lettura. Il file `foo` risulta invece `-rw-rwSr--`: proprietario e gruppo hanno lettura e scrittura, gli altri hanno sola lettura; la `S` maiuscola al posto della `x` del gruppo indica che il **bit SGID** è impostato mentre il bit di esecuzione per il gruppo **non** è attivo (se fosse attivo vedresti una `s` minuscola). È importante notare che il flag SGID rimane memorizzato anche quando non è presente la `x` (da qui la `S` maiuscola), ma lo SGID ha effetto—ossia, quando il file viene eseguito il processo eredita il GID del file—solo se il file è eseguibile e il sistema non impedisce l'uso dei bit speciali (ad es.

mount con `nosuid`, oppure il comportamento del kernel che ignora SUID/SGID su alcuni script interpretati). In sintesi: i permessi mostrati sono coerenti con le modifiche fatte dal programma; lo SGID è presente su `foo` e resterà inattivo finché non si rende eseguibile (o finché non vengono rispettate le condizioni che ne consentono l'applicazione).

System call `chown`, `fchown` e `lchown`

```
#include <unistd.h>

int chown (const char *path, uid_t owner, gid_t group);
int fchown (int fd, uid_t owner, gid_t group);
int lchown (const char *path, uid_t owner, gid_t group);
```

Le system call `chown()`, `fchown()` e `lchown()` servono per **modificare il proprietario e il gruppo** associati a un file nel sistema Unix.

La funzione `chown()` lavora sul file identificato da un **percorso** (path), mentre `fchown()` modifica le proprietà di un file **già aperto**, utilizzando il suo **file descriptor**. Invece, `lchown()` è simile a `chown()`, ma si applica direttamente a un **link simbolico**, senza seguire il collegamento al file a cui punta.

Tutte queste funzioni restituiscono **0 se l'operazione ha successo e -1 in caso di errore**.

Se si passa il valore **-1** come parametro per **owner** o **group**, significa che **quell'identificatore non deve essere modificato**. Ad esempio, se si vuole cambiare solo il gruppo, si può passare **-1** come `owner`, lasciando invariato il proprietario.

Solo un **processo con i privilegi del superutente (root)** può cambiare lo **user ID** di un file, cioè il proprietario. Un **utente normale** non può mai cambiare il proprietario di un file, neanche se quel file è suo.

Tuttavia, se il processo **non è root**, ma è **proprietario del file** (cioè il suo **effective user ID** e **real user ID** coincidono con lo **user ID del file**), allora **può cambiare il group ID** del file, ma **solo se il nuovo gruppo è uno di quelli a cui l'utente appartiene**, cioè deve far parte dei suoi **group ID supplementari**.

Dimensione dei file

```
struct stat
{
    dev_t      st_dev;      /* device */
    ino_t      st_ino;      /* inode */
    mode_t     st_mode;     /* file type & protection */
    nlink_t    st_nlink;    /* number of hard links */
    uid_t      st_uid;      /* user ID of owner */
    gid_t      st_gid;      /* group ID of owner */
    dev_t      st_rdev;     /* device type (if inode device) */
    off_t      st_size;     /* total size, in bytes */
    unsigned long st_blksize; /* blocksize for filesystem I/O */
    unsigned long st_blocks; /* number of blocks allocated */
    time_t     st_atime;     /* time of last access */
    time_t     st_mtime;     /* time of last modification */
    time_t     st_ctime;     /* time of last change */
};
```

La dimensione di un file, ovvero il numero totale di byte in esso contenuti, è memorizzata all'interno del campo **st_size** della struttura **stat**. Questo campo assume significato solo per **file regolari**, **directory** e **link simbolici**. In alcuni sistemi, come **Solaris**, viene assegnata una dimensione anche ai file di tipo **pipe**, intesa come il numero di byte attualmente disponibili per la lettura.

Oltre a `st_size`, la struttura `stat` include anche i campi `st_blksize` e `st_blocks`, che forniscono informazioni supplementari utili per l'efficienza delle operazioni di I/O. Il campo `st_blksize` rappresenta il miglior fattore di blocco da usare per accedere al file, mentre `st_blocks` indica il numero effettivo di blocchi da **512 byte** che sono stati allocati per il file nel filesystem.

La **libreria standard del linguaggio C**, nelle sue operazioni su file, tiene conto di queste informazioni per ottimizzare le prestazioni, scegliendo, ad esempio, buffer di dimensioni coerenti con `st_blksize`.

Troncamento: System call `truncate()` e `ftruncate()`

```
#include <unistd.h>

int truncate (const char *path, off_t length);
int ftruncate (int fd, off_t length);
```

Le system call **`truncate`** e **`ftruncate`**, definite nella libreria `<unistd.h>`, permettono di modificare la dimensione di un file già esistente, riducendola o estendendola al numero di **byte** specificato nel parametro **`length`**.

Quando la dimensione attuale del file è **maggiore** di `length`, tutte le informazioni memorizzate oltre tale soglia vengono **perse** e non sono più accessibili. In questo caso, il file viene effettivamente troncato.

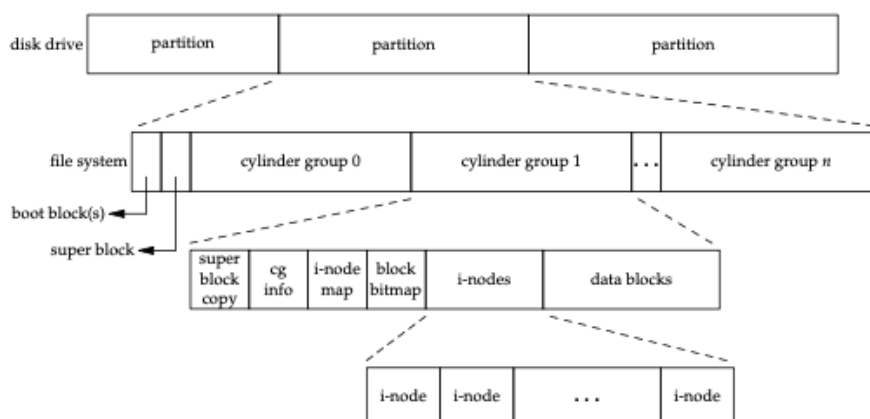
Nel caso opposto, se il file è **più corto** della lunghezza indicata, il comportamento della funzione **dipende dall'implementazione** del sistema operativo. Ad esempio, secondo lo **standard XSI (X/Open System Interface)**, il file viene esteso e i nuovi byte aggiunti sono **riempiti con zeri**.

Entrambe le funzioni restituiscono **0** in caso di successo, mentre restituiscono **-1** se si verifica un errore.

Queste operazioni sono utili in contesti in cui è necessario **tagliare** un file (ad esempio per rimuovere contenuto superfluo) o **preallocare spazio** senza ancora scrivere dati reali.

File System

Il **file system** è la componente del sistema operativo responsabile dell'organizzazione, memorizzazione, recupero e gestione dei dati su un supporto fisico come un hard disk.

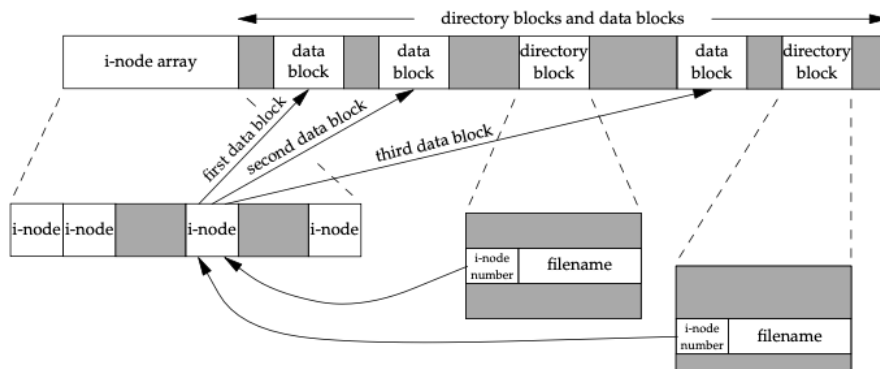


Nel caso di UNIX (UFS, UNIX File System), derivato da BSD, il disco fisico può essere suddiviso in più **partizioni**, e ciascuna partizione può contenere un intero file system.

All'interno di un file system, i dati sono organizzati in **cylinder group**, ciascuno dei quali include:

- una **copia del superblock**, che contiene le informazioni generali sul file system;
- una **mappa dei blocchi liberi** e una **mappa degli inode**;
- gli **inode**, che sono strutture dati che descrivono i file (contengono metadati ma non i nomi dei file);
- i **blocchi di dati**, in cui sono memorizzati i contenuti effettivi dei file.

Struttura e organizzazione



I file sono identificati da **nomi** nelle directory, ma il collegamento reale è tra un nome e un **inode number**, che punta a un inode nella struttura dati.

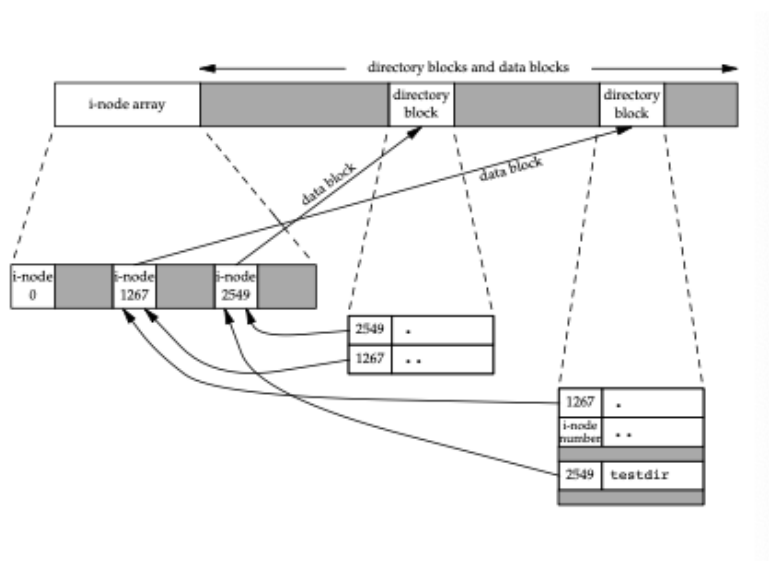
Un inode memorizza informazioni come:

- permessi di accesso,
- dimensione del file,
- UID e GID del proprietario,
- timestamp di accesso/modifica/cambio stato,
- puntatori ai **blocchi di dati**.

Una directory è a sua volta un file, composto da una lista di coppie (*nome del file*, *inode number*). Questo permette la gestione efficiente di collegamenti e percorsi nel file system.

Il collegamento tra inode e blocchi di dati può essere diretto (tramite puntatori diretti), oppure indiretto (con livelli di indirizzamento per file di grandi dimensioni).

Numero di link sulle directory



L'immagine rappresenta la situazione nel file system dopo aver creato una nuova directory chiamata **testdir** usando il comando:


```
mkdir testdir
```

Quando in un sistema Unix si procede alla **creazione di una nuova directory**, ad esempio attraverso il comando `mkdir`, il file system esegue una serie di operazioni coordinate. In primo luogo, viene generato un **nuovo inode**, il quale viene configurato per rappresentare una directory, includendo le informazioni di tipo, permessi e gli eventuali riferimenti ai blocchi dati iniziali. Successivamente, viene allocato almeno un **blocco di dati** associato a tale inode, nel quale vengono memorizzate le **voci di contenuto** della directory. Ogni directory, appena creata, contiene almeno due voci: `.` e `..`. La prima (`.`) è un collegamento che fa riferimento allo **stesso inode della directory**, mentre la seconda (`..`) punta all'**inode della directory padre**.

Parallelamente a queste operazioni, il contenuto della directory padre viene **modificato** per includere una **nuova voce** che associa il **nome della directory appena creata** al relativo **inode**. In questo modo, il nome simbolico (nel caso dell'esempio, "testdir") viene registrato all'interno del blocco dati della directory genitore, consentendo al sistema di risolvere il nome nel relativo inode tramite una semplice ricerca.

Nel modello Unix, è fondamentale comprendere che una **directory non è altro che un file speciale** contenente una lista di **associazioni tra nomi e inode**. Pertanto, il nome di un file o di una directory rappresenta un **riferimento simbolico**, e non un attributo intrinseco dell'inode stesso. L'astrazione fornita dagli inode consente un elevato grado di **flessibilità**, come dimostrato dalla possibilità di avere **link multipli** (hard link) che condividono lo stesso inode pur essendo accessibili attraverso nomi distinti in percorsi diversi.

I-node

Nel contesto dei sistemi Unix, il **file system** è strutturato attorno al concetto di **inode** (abbreviazione di *index node*), una struttura dati fondamentale che contiene tutte le **informazioni di metadati** associate a un file o a una directory, **escludendo esplicitamente il nome**. Ogni inode memorizza dati quali il **tipo dell'oggetto** (file regolare, directory, link simbolico, ecc.), i **permessi di accesso**, l'**identità del proprietario** e del **gruppo**, la **dimensione del file**, le **marcature temporali** (data di creazione, ultima modifica e ultimo accesso), nonché una serie di **puntatori** che indirizzano i **blocchi di dati** fisicamente allocati sul disco.

Link hard e simbolici

```
struct stat
{
    dev_t      st_dev;          /* device */
    ino_t      st_ino;          /* inode */
    mode_t     st_mode;         /* file type & protection */
    nlink_t    st_nlink;        /* number of hard links */
    uid_t      st_uid;          /* user ID of owner */
    gid_t      st_gid;          /* group ID of owner */
    dev_t      st_rdev;         /* device type (if inode device) */
    off_t      st_size;         /* total size, in bytes */
    unsigned long st_blksize;    /* blocksize for filesystem I/O */
    unsigned long st_blocks;     /* number of blocks allocated */
    time_t     st_atime;        /* time of last access */
    time_t     st_mtime;        /* time of last modification */
    time_t     st_ctime;        /* time of last change */
};
```

Nella struttura `stat`, il campo `st_nlink` rappresenta il **numero di hard link** associati a un file. Un **hard link** è un riferimento diretto all'i-node del file: ogni link aggiuntivo aumenta il valore di `st_nlink`, perché punta allo stesso contenuto fisico sul disco. Quando il valore di `st_nlink` scende a zero, significa che non esistono più riferimenti al file, e quindi lo spazio su disco può essere recuperato.

Diversamente, un **link simbolico** (o **symlink**) è un file speciale che contiene il **path** verso un altro file. Non modifica `st_nlink` perché non punta direttamente all'i-node, ma crea un riferimento "indiretto" al file.

Il campo `st_nlink` è dunque fondamentale per gestire correttamente la **condivisione di file tramite hard link**, monitorando quanti nomi di file (in diverse directory, ad esempio) condividono la stessa struttura dati fisica.

System call link() e unlink()

```
#include <unistd.h>

int link (const char *oldpath, const char *newpath);

    Ritorna 0 se OK, -1 in caso di errore
```

La **system call link()** consente di creare un **hard link** a un file esistente, ovvero un nuovo nome che fa riferimento allo stesso **i-node**. In questo modo, più nomi (in directory eventualmente diverse) possono riferirsi allo stesso contenuto fisico su disco, condividendo la stessa struttura di controllo.

Restituisce **0** in caso di successo e **-1** in caso di errore. Se `newpath` esiste già, viene generato un errore. Inoltre, molte implementazioni richiedono che **oldpath e newpath risiedano nello stesso file system**, anche se lo standard POSIX.1 prevede la possibilità futura di estendere il linking tra file system diversi.

Solo il **superutente** può creare hard link a directory, poiché ciò potrebbe compromettere la coerenza strutturale dell'albero delle directory. In generale, questa operazione è riservata a scopi specifici del sistema.

```
#include <unistd.h>

int unlink (const char *pathname);

    Ritorna 0 se OK, -1 in caso di errore
```

La **system call unlink()** rimuove una voce da una directory, eliminando il collegamento tra un nome e l'i-node corrispondente.

Restituisce **0** in caso di successo, **-1** in caso di errore. Quando un file viene "scollegato" tramite `unlink()`, il sistema decrementa il **contatore di link (link count)** presente nell'i-node. Finché questo contatore è maggiore di **0**, il file rimane accessibile. Tuttavia, quando scende a **0** e **nessun processo ha più il file aperto**, il contenuto del file può essere fisicamente rimosso dal disco.

È importante notare che l'operazione di `unlink()` richiede **permessi di scrittura ed esecuzione sulla directory** che contiene il file, non sul file stesso.

System call symlink() e readlink()

```
#include <unistd.h>

int symlink (const char *oldpath, const char *newpath);

int readlink (const char *path, char *buf, size_t bufsiz);
```

La system call **symlink()** consente di creare un **link simbolico**, ovvero un file speciale che contiene un **percorso** a un altro file o directory.

Dove `oldpath` è il file di destinazione e `newpath` è il nome del nuovo link simbolico. La funzione restituisce **0 in caso di successo e -1 in caso di errore**.

Per leggere il contenuto di un link simbolico (cioè il percorso memorizzato), si utilizza la system call **readlink()**,

Questa funzione **non apre il file di destinazione**, ma accede direttamente al contenuto del link simbolico (**percorso testuale** verso il file). Copia fino a `bufsiz` byte nella stringa `buf` (che **non viene terminata con \0**) e restituisce **il numero di byte letti**, oppure **-1 se si verifica un errore**.

System call mkdir() e rmdir()

```
#include <sys/stat.h>

int mkdir (const char *pathname, mode_t mode);

#include <unistd.h>

int rmdir (const char *pathname);
```

mkdir() e **rmdir()** sono due system call fondamentali per la **gestione delle directory** in ambiente Unix.

La funzione **mkdir()** permette di creare una nuova directory nel file system. Richiede come argomenti il **percorso** (*pathname*) della directory da creare e un valore *mode_t* che specifica i **permessi di accesso** della directory (come lettura, scrittura ed esecuzione per utente, gruppo e altri). Questo *mode* viene combinato con la *umask* del processo per determinare i permessi effettivi.

La funzione **rmdir()** consente invece di eliminare una directory esistente, a condizione che essa sia **vuota**, ovvero che non contenga altri file o sottodirectory (ad eccezione delle entry speciali *.* e *..*).

Entrambe le funzioni **ritornano 0 in caso di successo**, oppure **-1 in caso di errore**, nel qual caso *errno* viene impostato per indicare la causa (ad esempio: directory già esistente, permessi insufficienti, directory non vuota, ecc.).

Lecture delle directory

Una **directory** è un tipo speciale di file che contiene riferimenti ad altri file o directory. Per accedere ai suoi contenuti, è necessario avere i **permessi di lettura**, che consentono di visualizzarne la lista interna, ovvero i nomi dei file in essa contenuti. Tuttavia, avere il **permesso di scrittura o esecuzione** non implica necessariamente la possibilità di leggere i file elencati.

Nel linguaggio C, la lettura dei contenuti di una directory si effettua tramite le funzioni della libreria *<dirent.h>*. La funzione **DIR *opendir(const char *name)** apre la directory specificata dal parametro *name* e restituisce un puntatore a una struttura *DIR*. Questo puntatore viene poi passato alla funzione **struct dirent *readdir(DIR *dir)**, che permette di leggere sequenzialmente le entry della directory, ciascuna rappresentata da una struttura *struct dirent*.

```
struct dirent {
    ino_t d_ino;
    char d_name[NAME_MAX + 1];
};
```

In essa, il campo **d_name** contiene il nome del file o della directory.

Per tornare all'inizio della directory aperta si utilizza **rewinddir(DIR *dir)**, mentre **closedir(DIR *dir)** consente di chiudere la directory e liberare le risorse associate. Queste funzioni permettono un accesso sistematico al contenuto di una directory, analogamente a come si leggerebbe da un file sequenziale.

Esempio: elencare i file di una directory

```
#include <sys/types.h>
#include <dirent.h>
int main(int argc, char *argv[ ])
{
    DIR *dp;
    struct dirent *dirp;
    if (argc != 2){
        printf("a single argument (the directory name) is required");
        exit(-1);
    }
    if ( (dp = opendir(argv[1])) == NULL){
        printf("can't open %s", argv[1]);
        exit(-1);}
    while ( (dirp = readdir(dp)) != NULL)
        printf("%s\n", dirp->d_name);
    closedir(dp);
    exit(0);
}
```

Questo codice mostra come **elencare i file contenuti in una directory** utilizzando le funzioni della libreria `<dirent.h>`.

Il programma attende come **argomento della riga di comando** il nome della directory da aprire. All'inizio, controlla che venga fornito esattamente un argomento (`argc != 2`); in caso contrario, termina stampando un messaggio d'errore.

Successivamente, la funzione **`opendir(argv[1])`** tenta di aprire la directory specificata. Se il puntatore restituito è `NULL`, il programma stampa un errore e termina. Altrimenti, si entra in un ciclo `while` che utilizza **`readdir(dp)`** per leggere ogni entry della directory.

Ogni entry è una struttura `struct dirent` il cui campo **`d_name`** contiene il nome del file o della sottodirectory. Questo nome viene stampato a schermo.

Infine, il programma chiude la directory con **`closedir(dp)`** e termina correttamente.

System call `chdir()` e funzione `getcwd()`

```
#include <unistd.h>

int chdir (const char *path);
    Ritorna 0 se OK, -1 in caso di errore
char *getcwd (char *buf, size_t size);
    Ritorna puntatore a buf se OK, NULL in caso di errore
```

Ogni processo in esecuzione possiede una **directory di lavoro corrente** (detta anche **working directory**), che costituisce il punto di riferimento per l'interpretazione dei **path relativi**. Quando un processo accede a file o directory specificandone un **percorso relativo**, questo viene risolto a partire dalla sua directory di lavoro corrente.

Per conoscere quale sia la directory corrente di un processo, si utilizza la funzione **`getcwd`**, che restituisce il **percorso assoluto** della directory corrente, memorizzandolo nel **buffer** indicato dal primo argomento. Il secondo argomento specifica la **dimensione del buffer**. In caso di successo, la funzione restituisce un **puntatore al buffer**; in caso di errore, restituisce **`NULL`**.

Se si desidera modificare la directory corrente del processo, è possibile farlo mediante la system call **chdir**, che riceve in ingresso il **path della nuova directory**. In caso di successo, `chdir` restituisce **0**; in caso di errore, **-1**. Una volta modificata la directory corrente, tutti i **percorsi relativi** utilizzati successivamente dal processo verranno interpretati rispetto alla nuova directory impostata tramite `chdir`. Questa operazione è concettualmente equivalente al comando **cd** utilizzato nella **shell** per cambiare directory.

Esempio `getcwd` – Ottenere la directory corrente

```
#include <stdio.h>
#include <unistd.h>
#include <limits.h>

int main() {
    char cwd[PATH_MAX];

    if (getcwd(cwd, sizeof(cwd)) != NULL) {
        printf("Directory corrente: %s\n", cwd);
    } else {
        perror("getcwd");
    }

    return 0;
}
```

Questo programma stampa la directory corrente del processo, equivalente al comando `pwd` in shell.

Esempio `chdir` – Cambiare la directory corrente

```
#include <stdio.h>
#include <unistd.h>
#include <limits.h>

int main() {
    char cwd[PATH_MAX];

    // Cambio directory
    if (chdir("/tmp") == 0) {
        printf("Directory cambiata con successo.\n");

        // Verifico dove mi trovo adesso
        if (getcwd(cwd, sizeof(cwd)) != NULL) {
            printf("Nuova directory corrente: %s\n", cwd);
        }
    } else {
        perror("chdir");
    }

    return 0;
}
```

Questo programma cambia la directory corrente a `/tmp`, e poi la stampa per verificarlo.